

RPGXchange

One format. Infinite adventures.

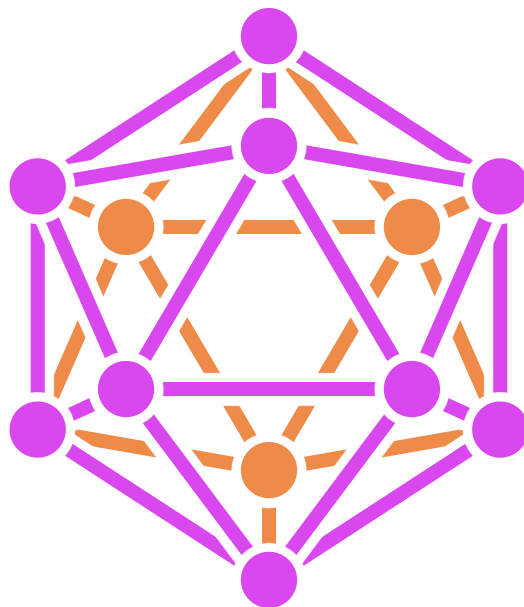


Table of contents:

RPGXchange v0.2.0-next

File Structure Overview

- [Basic Structure](#)
- [Assets Directory](#)
- [Data Directory](#)
- [Best Practices](#)
- [Version Control](#)
- [File Extensions](#)

Metadata

- [Properties](#)
 - [Example](#)
- [Version Support](#)

Variables

- [Location](#)
- [Structure](#)
 - [Properties](#)
 - [Example](#)
- [Implied Variable Definition](#)
 - [Implied Definition Syntax](#)
 - [Implied Definition Example](#)
- [Value Field](#)
 - [Type Validation](#)
 - [String Interpolation](#)
 - [Example](#)
- [Best Practices](#)
- [Version Support](#)

Functions

- [Location](#)
- [Structure](#)
 - [Properties](#)
 - [Example](#)
- [Version Support](#)

Namespaces

- [Reserved Directory Names](#)
- [Namespace Path Specification](#)

- Example Structure:
- Version Support

Data Types

- Available Types
 - Integer
 - Number
 - String
 - Boolean
 - Array
 - Object
 - Null
- Type Strings
- Type Coersion
- Version Support

Formulas

- Basic Syntax
- Operators
- Operator Precedence
- Variable References
- Built-in Functions
- Error Handling

Resource IDs

Versioning

- File Versioning
 - Version Selectors

Platform Compatibility

- Platform Compatibility
 - Compatibility Format
 - Compatibility Behavior
- Implementation Requirements
 - File Loading
 - Resource Processing
 - Best Practices
- Version Support

License

- Branding and Trademark
- Code and Documentation License

RPGXchange v0.2.0-next

RPGXchange (RPGX) is an open specification for a versatile data interchange format designed to bridge the gap between various tabletop roleplaying game (TTRPG) platforms, tools, and applications. This format enables seamless transfer of character data, game content, and resources across virtual tabletops (VTTs), character managers, digital assistants, and other gaming platforms.



WARNING

This version of RPGXchange is a development version and incomplete. While we encourage exploration and feedback, we recommend against using it in production environments.

File Structure Overview

RPGXchange (RPGX) files are ZIP archives containing structured data and assets for tabletop roleplaying games. This document outlines the standard file structure for both RPGXSource (RPGXS) and RPGXCharacter (RPGXC) files.

Basic Structure

Located within the root of the archive file, is a `metadata.json` file that contains information about the RPGX file. There is also optionally a `data` folder and an `assets` folder to contain the contents of the RPGX file. Some platforms or clients may define additional directories or files for additional functionality.

```
rpgx-file.rpgx/  
├ assets/  
├ data/  
└ metadata.json
```

Assets Directory

The `assets/` directory contains all media files used by the RPGX file. Assets should be specified by their path relative to the assets folder.

Data Directory

The `data/` directory contains all game-related data.

Best Practices

1. Use clear, descriptive namespaces

2. Keep file names short but meaningful
3. Group related content in subdirectories
4. Use platform-specific overrides sparingly
5. Maintain backward compatibility when possible
6. Document any custom implementations
7. Follow JSON schema definitions
8. Include appropriate validation rules
9. Implement proper version control

Version Control

RPGX files follow [semantic versioning](#) (MAJOR.MINOR.PATCH) conventions.

More specific versioning details can be found on [Versioning](#).

File Extensions

- `.rpgxs`: RPGXSource files
- `.rpgxc`: RPGXCharacter files
- `.rpgx`: Generic RPGXchange files

Metadata

The `metadata.json` file is required in the root directory and contains essential information about the RPGX file.

Properties

Property	Type	Required	Default	Description
id	string	Yes	-	The semi-unique identifier for the RPGX file; Must follow resource id rules
name	string	No	Value of <code>id</code>	The user friendly name of the RPGX file
description	string	No	-	The user friendly description of the RPGX file
version	string	No	-	The version of the RPGX file in the format of X.Y.Z with the Y and Z sections being optional; See Versioning
type	string	No	<code>"source"</code> for RPGXS;	The type of RPGX file: either

Property	Type	Required	Default	Description
			"character" for RPGXC	"source" or "character"
namespace	string	No	-	The root namespace of the RPGX file
authors	array	No	[]	A list of authors of the project. Can be strings or objects with name, email, url, and/or credit fields
dependencies	array	No	[]	A list of required RPGX files. Can be strings or objects with id, and version selector
rpgxVersion	string	No	-	The target version of RPGX to use. See Versioning
platforms	array	No	-	List of supported platforms; See Platform Compatibility

Example



```

{
  "id": "example",
  "name": "Example RPGX File",
  "description": "A comprehensive example of an RPGX file
structure",
  "version": "1.0.0",
  "type": "source",
  "namespace": "example",
  "authors": [
    {
      "name": "John Doe",
      "email": "john@example.com",
      "url": "https://example.com/johndoe",
      "credit": "Creative design"
    },
    "Jane Smith"
  ],
  "dependencies": [
    {
      "id": "core",
      "version": "1.0"
    },
    "fantasy"
  ],
  "rpgxVersion": "0.1.0"
}

```

Version Support

Version	Support Level	Notes
0.1.0	 Partial Support	Support for id, name, description, version, namespace, authors, dependencies, and rpgxVersion
0.2.0	 Support	Added support for platform compatibility specifications

Variables

Variables are values that are stored within a RPGX file inside of `variables.json` files located in the ZIP archive. These files must be located within the `data` folder in a directory structure that corresponds to their [namespace](#).

Location

Variables must be organized in `variables.json` files within the `data` directory. The path to the file should match the namespace of the variables. For example:

- Variables in namespace `combat:stats` should be in `/data/combat/stats/variables.json`
- Variables in namespace `character` should be in `/data/character/variables.json`

Structure

Variables are defined as JSON objects with specific properties that determine their behavior and characteristics.

Properties

Property	Type	Required	Default	Description
type	string	No	<code>"any"</code>	The data type of the variable; See Data Types
description	string	No	-	A user-friendly description of what the

Property	Type	Required	Default	Description
				variable represents
value	varies	No	<code>null</code>	The value of the variable
platforms	array	No	undefined	List of supported platforms; See Platform Compatibility

Example

```
{
  "strength": {
    "type": "integer",
    "description": "The physical strength attribute of the character",
    "value": 10,
    "platforms": [
      {
        "id": "rpgxjs",
        "version": "1.0.0"
      }
    ]
  },
  "bonus": {
    "type": "number",
    "description": "Calculated bonus damage based on strength",
    "value": "{floor(character:stats:strength / 2)}"
  },
  "isAlive": {
    "type": "boolean",
    "description": "Indicates if the character is currently alive",
    "value": true
  },
  "title": {
    "type": "string",
    "description": "The character's earned title",
    "value": "Novice"
  }
}
```

```
}  
}
```

Implied Variable Definition

For simplicity, variables can also be defined using an implied definition where the value is specified directly. This is equivalent to defining a variable with only the `value` property. It's particularly useful when assigning values to variables in a character file where the variable has already been defined in a source file.

Implied Definition Syntax

```
{  
  "variableName": value  
}
```

This implied definition is valid for all data types except `object` to avoid potential ambiguity between an object that represents a variable definition and an object that is the actual value of the variable.

Implied Definition Example

```
{  
  "strength": 10,  
  "intelligence": 12,  
  "isAlive": true,  
  "title": "Novice",  
  "skills": ["stealth", "archery", "diplomacy"]  
}
```

This is equivalent to:

```
{  
  "strength": {  
    "value": 10  
  },  
}
```

```

"intelligence": {
  "value": 12
},
"isAlive": {
  "value": true
},
"title": {
  "value": "Novice"
},
"skills": {
  "value": ["stealth", "archery", "diplomacy"]
}
}

```

Value Field

The `value` field can contain either:

- A literal value matching the specified type
- A formula string enclosed in curly braces `{}`

Type Validation

Values are validated against their declared type:

Type	Valid Values	Examples
<code>integer</code>	Whole numbers	<code>5</code> , <code>{floor(3.7)}</code>
<code>number</code>	Any numeric value	<code>3.14</code> , <code>{character:abilities:strength * 1.5}</code>
<code>string</code>	Text strings	<code>"Hello"</code> , <code>{name + " " + title}</code>
<code>boolean</code>	True/false values	<code>true</code> , <code>{health > 0}</code>

Type	Valid Values	Examples
array	List of values	<code>[]</code> , <code>[1,2,3]</code> , <code>[[], [1], [2]]</code>
object	Key-value pairs	<code>{}</code> , <code>{"x":1}</code> , <code>{"a":{"b":2}}</code>
null	Null value only	<code>null</code>
any	Any valid JSON value	<code>42</code> , <code>"text"</code> , <code>true</code>

String Interpolation

String values can include embedded expressions using curly braces `{}`. These expressions are evaluated and their results are inserted into the string.

To include literal curly braces in a string, use double curly braces `{{}}`.

Example

```
{
  "greeting": {
    "type": "string",
    "value": "Hello, {character:name}!" // Evaluates
character:name
  },
  "status": {
    "type": "string",
    "value": "HP: {{current}} / {maxHealth}" // Shows "
{current}" literally and evaluates maxHealth
  }
}
```

Best Practices

1. Use descriptive variable names
2. Keep namespaces organized

3. Consider variable dependencies
4. Use type coercion sparingly
5. Use implied variable definitions when appropriate for cleaner character files

Version Support

Version	Support Level	Notes
0.1.0	 Partial Support	Support for integer, number, string, boolean, and null
0.2.0	 Support	Added array and object data types, implied variable definitions, platform compatibility specifications

Functions

Functions are operations that are defined within RPGX files inside of `functions.json` files located in the ZIP archive. These files must be located within the `data` folder in a directory structure that corresponds to their [namespace](#).

Location

Functions must be organized in `functions.json` files within the `data` directory. The path to the file should match the namespace of the functions. For example:

- Functions in namespace `combat.stats` should be in `/data/combat/stats/functions.json`
- Functions in namespace `character` should be in `/data/character/functions.json`

Structure

Functions are defined as JSON objects with specific properties that determine their behavior and characteristics.

Properties

Property	Type	Required	Default	Description
parameters	object	No	<code>{}</code>	Variable definitions for the parameters of the function

Property	Type	Required	Default	Description
body	formula string	Yes	-	An expression for the calculation of the function
returnType	string	No	"any"	The data type of the variable; See Data Types
description	string	No	""	A user-friendly description of what the variable represents
platforms	array	No	undefined	List of supported platforms; See Platform Compatibility

Example

```
{
  "calcModifier": {
    "parameters": {
      "abilityScore": {
        "type": "integer"
      }
    },
    "body": "{floor((abilityScore - 10) / 2)}",
    "returnType": "integer",
    "description": "Calculates the modifier score for a given ability score"
  }
}
```

Version Support

Version	Support Level	Notes
0.1.0	✗ Not Supported	
0.2.0	✓ Supported	Initial implementation of functions

Namespaces

Namespaces in RPGX files are determined by the resource path within rooted directories (e.g., `data` or `assets`) inside the ZIP archive. The namespace hierarchy follows folder paths, with each subdirectory representing a segment of the namespace. Every namespace is prepended by the namespace specified in the `metadata.json` for the RPGX file.

Reserved Directory Names

Some folder names are reserved and do not contribute to namespace paths. These folders are designated for containing files of specific functionality and purpose.

Namespace Path Specification

Namespaces are specified using the path with colons (`:`) as segment separators. When referencing a namespace, the system will match to the closest matching namespace segment that aligns with the provided path. For instance, if referencing `combat:spells`, the closest matching subnamespace to the current reference will be used.

To avoid ambiguity in namespace resolution, it's recommended to use absolute paths that include all namespace segments from the root. This ensures precise and predictable namespace matching.

Example Structure:

```
/           # Base namespace from metadata.json:
example
└─ data/    # Reserved directory
    └─ combat/ # Namespace: example:combat
```

```
| └─ spells/      # Namespace: example:combat:spells
|   └─ weapons/   # Namespace: example:combat:weapons
└─ characters/    # Namespace: example:characters
```

Version Support

Version	Support Level	Notes
0.1.0	✓ Support	Initial implementation
0.2.0	✓ Support	Changed namespace delimiter from <code>.</code> to <code>:</code>

Data Types

There are various data types available for values within an RPGX file. The available data types are integer, number, string, boolean, array, object, and null.

Available Types

Integer

Integers represent whole numbers without any decimal points. Example: `42`, `-17`, `0`

Number

Numbers represent both integer and floating-point values. Example: `3.14`, `-0.001`, `2.5e-4`

String

Strings are sequences of characters enclosed in double quotes. They can contain any Unicode characters. Example: `"Hello"`, `"Player 1"`, `"🎮"`

Boolean

Booleans represent logical values: true or false. Example: `true`, `false`

Array

Arrays are ordered collections of values, which can be of any valid data type including other arrays. Example: `[1, 2, 3]`, `["red", "green", "blue"]`, `[true, 42, "mixed", [1, 2]]`

Object

Objects are collections of key-value pairs, where keys are strings and values can be any valid data type. Example: `{"name": "Player", "level": 5, "active": true}, {"coordinates": {"x": 10, "y": 20}}`

Null

Null represents the absence of any value. Example: `null`

Type Strings

Type strings are a JSON string that represents a given data type. A type string can contain the name of any supported data type or the word `any` for any data type.

Type Coersion

When a value doesn't match the declared type:

1. Numbers are rounded to integers for `integer` type
2. Non-boolean values are converted to boolean (`0`, `""`, `{}`, `[]`, become `false`; `null` converts to `false` if value is non-nullable)
3. All other values are converted to strings when type is `string` (`undefined` and `null` are converted to empty string if value is non-nullable)
4. Individual values are wrapped in arrays when type is `array`
5. Arrays can be coerced to objects where the keys are the indices of the array and the values are the elements within the array
6. Arrays can be coerced to a single value if the array only contains a single value.
7. Invalid conversions result in `undefined`

Version Support

Version	Support Level	Notes
0.1.0	 Partial Support	Support for integer, number, string, boolean, and null
0.2.0	 Support	Added array and object support

Formulas

Formulas in RPGX allow dynamic computation of values. They can be used in variables, conditions, and other dynamic fields.

Basic Syntax

In non-dedicated fields, formulas are enclosed in curly braces. In fields expecting a formula, enclosing curly braces are not necessary.

Operators

Operator	Description	Example	Notes
<code>+</code>	Addition/concatenation	<code>1 + 2</code> , <code>"a" +</code> <code>"b"</code>	When either operand is a string, the other operand will be converted into a string; else, both values are converted to numbers when possible; otherwise, results in <code>undefined</code>
<code>-</code>	Subtraction	<code>5 - 3</code>	Both operands are converted to numbers when possible; otherwise, results in <code>undefined</code>
<code>*</code>	Multiplication	<code>4 * 2</code>	Both operands are converted to

Operator	Description	Example	Notes
			numbers when possible; otherwise, results in <code>undefined</code>
/	Division	<code>10 / 2</code>	Both operands are converted to numbers when possible; otherwise, results in <code>undefined</code>
%	Modulo	<code>7 % 3</code>	Both operands are converted to numbers when possible; otherwise, results in <code>undefined</code>
==	Equality	<code>x == y</code>	
!=	Inequality	<code>x != y</code>	
>	Greater than	<code>x > y</code>	If either value is not comparable, then <code>undefined</code> is returned
>=	Greater or equal	<code>x >= y</code>	If either value is not comparable, then <code>undefined</code> is returned
<	Less than	<code>x < y</code>	If either value is not comparable, then <code>undefined</code> is returned

Operator	Description	Example	Notes
<code><=</code>	Less or equal	<code>x <= y</code>	If either value is not comparable, then <code>undefined</code> is returned
<code>&&</code>	Logical AND	<code>x > 0</code> <code>&& y < 10</code>	Both operands are coerced into boolean values
<code> </code>		<code> </code>	Logical OR

Operator Precedence

Operators are evaluated in the following order (highest to lowest):

1. Parentheses `()`
2. Multiplication/Division/Modulo `* / %`
3. Addition/Subtraction `+ -`
4. Comparison `< <= > >=`
5. Equality `== !=`
6. Logical AND `&&`
7. Logical OR `||`

Variable References

Variables and other resources can be referenced using either its id or namespace and id. When referencing a variable with its namespace, use the namespace path followed by a colon followed by the resource id:

- Resource Id: `strength`
- Namespaced reference: `character:stats:strength`

Built-in Functions

Function	Description	Example
<code>floor(x)</code>	Round down	<code>floor(3.7)</code> -> 3
<code>ceil(x)</code>	Round up	<code>ceil(3.2)</code> -> 4
<code>round(x)</code>	Round to nearest	<code>round(3.5)</code> -> 4
<code>min(x,y)</code>	Minimum value	<code>min(5, 3)</code> -> 3
<code>max(x,y)</code>	Maximum value	<code>max(5, 3)</code> -> 5
<code>if(c,t,f)</code>	Conditional	<code>if(x>0, x, 0)</code>
<code>length(x)</code>	Gets the length of a string	<code>length("xyz")</code> -> 3

Error Handling

Any time an error occurs, `undefined` can be used to continue calculation. Scenarios that may warrant use of `undefined` include but are not limited to the following cases:

- Division by zero
- Invalid type conversions
- Missing variable references
- Invalid function arguments
- Comparison of incompatible types

Resource IDs

Resource IDs are semi-unique identifiers used for file IDs in the [metadata](#), [namespaces](#), and other resources. Resource IDs must adhere to the following rules:

- Must be alphanumeric (both uppercase and lowercase letters are allowed).
- Can contain underscores.
- Cannot start with a number.
- Cannot start or end with an underscore.

Versioning

File Versioning

RPGX files should include a version number that follows Semantic Versioning ([SemVer](#)) principles. The version number should be specified in the format

`MAJOR.MINOR.PATCH`:

- `MAJOR` version changes indicate incompatible modifications
- `MINOR` version changes add functionality in a backward-compatible manner
- `PATCH` version changes include backward-compatible bug fixes

For example, an RPGX file at version `1.2.3` indicates:

- Major version: 1
- Minor version: 2
- Patch version: 3

We strongly recommend using SemVer to ensure consistent version management and compatibility checking across RPGX files.

Version Selectors

Version selectors follow semantic versioning (SemVer) rules and can be expressed in several ways:

Selector	Description	Example
<code>=x.y.z</code>	Exact version match	<code>=1.2.3</code> matches only 1.2.3
<code>>x.y.z</code>	Greater than version	<code>>1.2.3</code> matches 1.2.4, 1.3.0, 2.0.0

Selector	Description	Example
<code>>=x.y.z</code>	Greater than or equal to	<code>>=1.2.3</code> matches 1.2.3, 1.2.4, 1.3.0
<code><x.y.z</code>	Less than version	<code><1.2.3</code> matches 1.2.2, 1.1.0
<code><=x.y.z</code>	Less than or equal to	<code><=1.2.3</code> matches 1.2.3, 1.2.2, 1.1.0
<code>x.y.z</code>	Exact version (= implied)	<code>1.2.3</code> matches only 1.2.3

Multiple selectors can be combined:

- Using spaces for AND: `>=1.2.0 <2.0.0`
- Using `||` for OR: `1.2.3 || >=1.3.0 <2.0.0`

Prerelease versions (e.g., `1.2.3-alpha.1`) are only included when explicitly specified in the version selector.

Platform Compatibility

RPGXchange supports platform-specific implementations through compatibility declarations. This allows content creators to specify which platforms their content is designed to work with and enables platform-specific features.

Platform Compatibility

Platform compatibility can be declared with different levels of granularity:

1. File Level - Using the `platforms` property in `metadata.json`
2. Resource Level - Using the `platforms` property in resource definitions (such as variables and functions)

Compatibility Format

Platform compatibility is specified using an array of platform identifiers or objects:

```
{
  "platforms": [
    "platform-id",
    {
      "id": "another-platform",
      "version": ">=1.0.0 <2.0.0"
    }
  ]
}
```

Compatibility Behavior

The following rules govern platform compatibility:

1. If `platforms` is empty, undefined, `null`, or unspecified:

- The content is treated as compatible with all platforms
 - No compatibility restrictions are applied
2. If `platforms` contains entries:
- Only listed platforms should process the content
 - Platforms not listed should ignore the content
 - Platform IDs are not necessarily unique - platforms may process content intended for other platforms if they provide compatible functionality

Implementation Requirements

File Loading

When loading RPGX files:

1. Check file-level platform compatibility in `metadata.json`
2. If the current platform is not listed in `platforms`:
 - Display a clear warning to the user about potential compatibility issues
 - Continue loading if possible, but flag content as potentially incompatible

Resource Processing

When processing resources:

1. Check platform compatibility for each resource
2. Skip any resource that specifies platforms but doesn't list the current platform
3. Process resources that either:
 - Don't specify platform restrictions
 - List the current platform as compatible
 - List a platform ID that the current implementation supports

Best Practices

1. Clearly document which platform IDs your implementation responds to
2. Provide clear feedback when skipping content due to platform restrictions
3. Consider supporting commonly used platform IDs if your implementation is compatible
4. Use semantic versioning for platform version requirements
5. Document any platform-specific features or behaviors

Version Support

Version	Support Level	Notes
0.1.0	✗ Not Supported	
0.2.0	✓ Support	Initial support

License

Branding and Trademark

The RPGXchange logo, favicon, and branding assets are **NOT** covered by the CC BY-SA 4.0 license. These assets are protected intellectual property.

See the [Branding Guidelines](#) for complete trademark usage terms.

Quick summary:

-  Use in documentation to show compatibility with the format
-  Use badges/logos to indicate format support
-  Do not modify the logo
-  Do not use in derivative projects claiming to be RPGXchange
-  Do not use in ways suggesting ownership or endorsement

[Full branding guidelines](#)

Code and Documentation License

All code and documentation in this repository are licensed under the **Creative Commons Attribution-ShareAlike 4.0 International** license.

This means you can:

- Use, modify, and distribute the code and documentation
- Create derivative works
- Use for commercial purposes

Provided that you:

- Give appropriate credit to RPGXchange
- License your derivative works under the same license

- Indicate what changes you made

For the complete legal text, see the [LICENSE file](#) in the repository root.